

Documento di Progetto

GuessTheWord Game

v1.0

Gruppo 5

Antonio Ametrano – Matr. 0612710145

Hermann Galluccio – Matr. 0612709328

Chiara Iacuzzo – Matr. 0612708797

Francesco Vecchione – Matr. 0612709314

Informazioni sul Progetto

Repository GitHub

Repository del progetto:

<https://github.com/antonioame/GuessTheWord>

Cronologia dei Contributi

I contributi per ogni membro del team sono revisionabili dalle storie dei commit accessibili tramite i seguenti link:

- Antonio Ametrano:
<https://github.com/antonioame/GuessTheWord/commits/main/?author=antonioame>
- Hermann Galluccio:
<https://github.com/antonioame/GuessTheWord/commits/main/?author=Shuzer07>
- Chiara Iacuzzo:
<https://github.com/antonioame/GuessTheWord/commits/main/?author=ciacuzzo5-ux>
- Francesco Vecchione:
<https://github.com/antonioame/GuessTheWord/commits/main/?author=francesco-vecchione>

Profili di debug

Per facilitare il test dell'applicazione sono stati creati i seguenti profili di default:

- Profilo del Server: Username: "s1", Password: "s1";
- Profilo del Client 1: Username: "p1", Password: "p1";
- Profilo del Client 2: Username: "p2", Password: "p2";

1 Architettura del Sistema

Il sistema adotta un'architettura client-server classica, distribuita su diversi nodi che comunicano tramite protocolli standard. La separazione delle responsabilità garantisce modularità e indipendenza tra l'interfaccia utente, la logica applicativa centrale e lo strato di persistenza dei dati.

Di seguito viene descritta la composizione dei singoli nodi e i relativi canali di comunicazione:

1. Nodi dell'architettura

- **Dispositivo Utente (Client)**

Rappresenta il nodo client a disposizione dell'utente finale. Ospita l'artefatto GTWClient.jar, ovvero l'applicazione eseguibile per il giocatore.

- **Server Centrale (Host)**

È il middleware o server di backend dell'infrastruttura. Ospita l'artefatto GTWServer.jar, il quale contiene la logica di business fondamentale del sistema. Questo nodo si occupa di elaborare le richieste provenienti dal client, coordinare le dinamiche dell'applicazione e interfacciarsi con il database.

- **ServerDB (Database)**

È il nodo dedicato alla persistenza dei dati. Su di esso sono definite le relazioni per la memorizzazione delle entità del dominio, tra cui Player, Match e Challenge.

2. Connessioni

- **Client - Host**

La comunicazione tra il dispositivo utente e il server centrale avviene a livello di trasporto tramite il protocollo TCP/IP. Lo scambio di dati applicativi è astratto e serializzato attraverso l'invio di oggetti/messaggi specifici denominati NetworkMessage, garantendo una comunicazione affidabile e orientata alla connessione.

- **Host - Database**

Il server centrale comunica con il database utilizzando la tecnologia JDBC (Java Database Connectivity). L'accesso ai dati è mediato dal pattern DAO (Data Access Object) implementato nel server, il quale astrae le query SQL consentendo l'operatività (CRUD) sulle tabelle definite nel database.

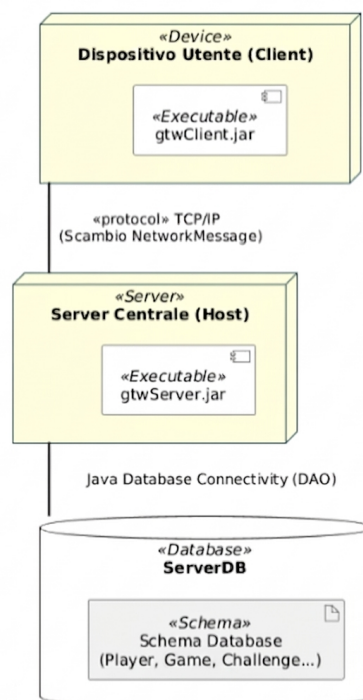


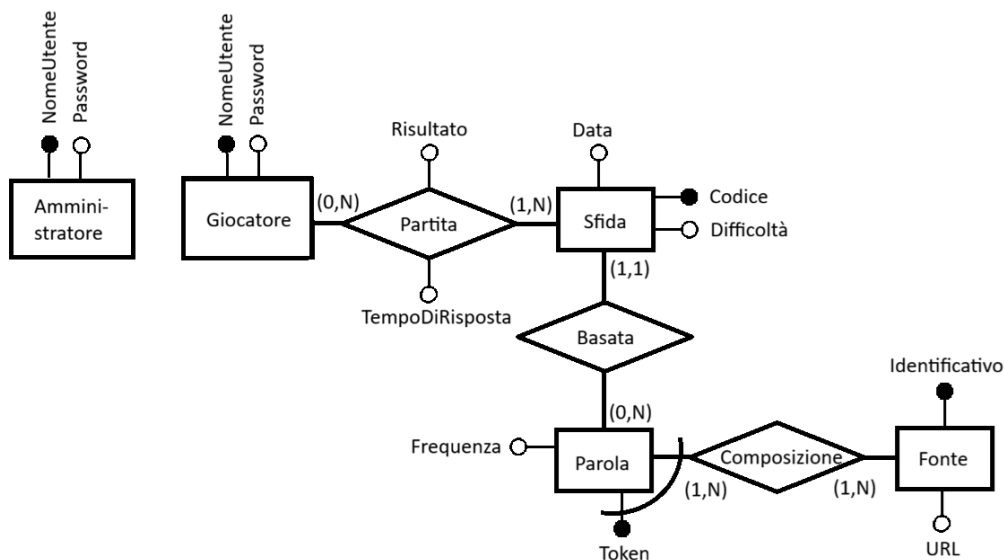
Diagramma di distribuzione

Diagramma di distribuzione

2 Progettazione Logica

2.1 Ristrutturazione preliminare

Si è deciso di eliminare la generalizzazione su Utente accorpendo il padre alle figlie in quanto questa generalizzazione è totale e l'accesso alle figlie (Amministratore e Giocatore) è distinto.



Schema E-R ristrutturato senza generalizzazioni o attributi composti

Concetto	Tipo	Volume
Amministratore	E	2
Giocatore	E	20
Sfida	E	10000
Parola	E	200000
Fonte	E	20
Partita	R	20000
Basata	R	9000
Composizione	R	20000

Tavola dei volumi

Seguendo la regola dell'80-20, il 20% delle operazioni che costituiscono l'80% del carico operativo sono le seguenti:

ID	Tipo	Frequenza
OP-1	B	100 al giorno
	Recuperare tutte le parole per una certa fonte.	
OP-2	B	100 al giorno

	Memorizzazione della partita svolta da due giocatori e della sfida associata.	
OP-3	I	100 al giorno
	Recuperare le statistiche di partite vinte, partite giocate e tempo medio per un certo giocatore.	
OP-4	I	1 al giorno
	Recuperare la classifica dei giocatori ordinati in base alle vittorie e al minor numero di partite giocate e al minor tempo medio.	

Tavola delle operazioni

2.2 Analisi delle ridondanze

Una considerazione preliminare che bisogna fare è quali operazioni possano effettivamente fare uso di ridondanze. Escludendo *OP-1* per la semplice ragione che necessita, in un modo o nell'altro, di fare accesso a tutte le parole appartenenti ad una certa fonte, l'analisi delle ridondanze si può concentrare sulle restanti tre operazioni.

Le ridondanze che si è ipotizzato di inserire nello schema sull'entità Giocatore sono le seguenti:

- Tempo totale giocato: calcolato in secondi, è la somma dei secondi impiegati per rispondere a ciascuna partita;
- Numero partite vinte;
- Numero partite giocate.

OP-2			
Concetto	Costrutto	Accessi	Tipo
Sfida	E	2	S
OP-3			
Concetto	Costrutto	Accessi	Tipo
Giocatore	E	1	L
Partita	R	500	L
OP-4			
Concetto	Costrutto	Accessi	Tipo
Partita	R	10000	L

Tavola degli accessi in assenza delle ridondanze

OP-2			
Concetto	Costrutto	Accessi	Tipo
Sfida	E	1	S
Partita	R	2	S
Giocatore	E	2	L
Giocatore	E	2	S

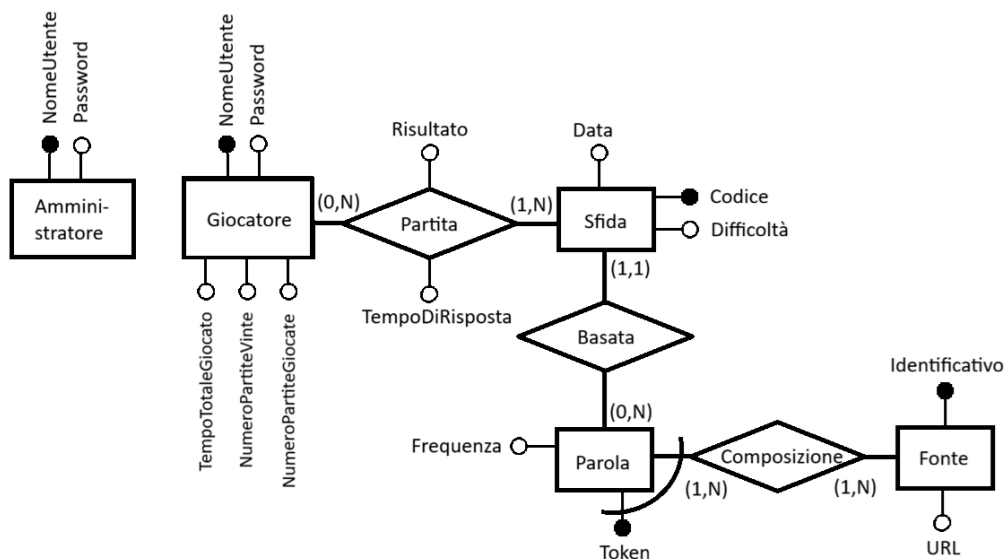
OP-3			
Concetto	Costrutto	Accessi	Tipo
Giocatore	E	1	L
OP-4			
Concetto	Costrutto	Accessi	Tipo
Giocatore	E	20	L

Tavola degli accessi in presenza delle ridondanze

Operazione	Costo in assenza ridondanze	Costo in presenza ridondanze
OP-2	$4 \cdot 100 = 400$	$(2+4+2+4) \cdot 100 = 1200$
OP-3	$(500+1) \cdot 100 = 50100$	$1 \cdot 100 = 100$
OP-4	$10000 \cdot 1 = 10000$	$20 \cdot 1 = 20$
Totali	circa 61400	circa 1320

Tavola dei costi

Senza alcun dubbio, dall'analisi delle ridondanze si evince che la scelta migliore sia quella di inserire le ridondanze che abbiamo ipotizzato all'inizio. Lo schema E-R ristrutturato con l'aggiunta delle ridondanze discusse sull'entità Giocatore sarà:



Schema E-R ristrutturato finale

2.3 Traduzione verso il modello logico

Per l'implementazione del database si è scelto di usare *SQLite* come RDBMS. Inoltre, per rendere universale l'implementazione, si è scelto di scrivere i nomi delle relazioni e degli attributi in inglese.

```
CREATE TABLE admin (  
    username          TEXT NOT NULL,  
    password          TEXT NOT NULL,  
  
    PRIMARY KEY(username)  
);
```

```
CREATE TABLE player (  
    username          TEXT NOT NULL,  
    password          TEXT NOT NULL,  
    totalPlayedTime   INTEGER DEFAULT 0,  
    totalGamesWon     INTEGER DEFAULT 0,  
    totalGamesPlayed  INTEGER DEFAULT 0,  
  
    PRIMARY KEY(username)  
);
```

```
CREATE TABLE game (  
    player            TEXT NOT NULL,  
    challenge         INTEGER NOT NULL,  
    result            TEXT NOT NULL,  
    responseTime      INTEGER NOT NULL,  
  
    PRIMARY KEY(player, challenge),  
  
    FOREIGN KEY(player)  
        REFERENCES player(username)  
        ON DELETE RESTRICT,  
  
    FOREIGN KEY(challenge)  
        REFERENCES challenge(code)  
        ON DELETE RESTRICT  
);
```

```
CREATE TABLE challenge (  
    code              INTEGER NOT NULL,  
    date              TEXT NOT NULL,  
    difficulty        TEXT NOT NULL,  
    word              TEXT NOT NULL,  
    source            INTEGER NOT NULL,  
  
    PRIMARY KEY(code),  
  
    FOREIGN KEY(word, source)  
        REFERENCES word(token, source)  
        ON DELETE RESTRICT  
);
```

```

CREATE TABLE word (
    token                TEXT NOT NULL,
    frequency            INTEGER DEFAULT 0,
    source                INTEGER NOT NULL,

    PRIMARY KEY(token, source),

    FOREIGN KEY(source)
        REFERENCES source(id)
        ON DELETE CASCADE
);

```

```

CREATE TABLE source (
    id                   INTEGER NOT NULL,
    path                 TEXT,

    PRIMARY KEY(id)
);

```

Per le relazioni appena create, è facile dimostrare che siano in BCNF in quanto tutti gli attributi dipendono da una (super)chiave della relazione.

Inoltre, per implementare la gestione automatica delle ridondanze sono previsti due trigger capaci di intercettare l'inserimento di una nuova partita e la cancellazione di una sorgente.

```

CREATE TRIGGER incrementTotalsInPlayer
AFTER INSERT ON game
FOR EACH ROW
BEGIN
    UPDATE player
    SET totalPlayedTime = totalPlayedTime + NEW.responseTime,
        totalGamesWon = totalGamesWon +
            CASE
                WHEN NEW.result = 'WIN' THEN 1
                ELSE 0
            END,
        totalGamesPlayed = totalGamesPlayed + 1
    WHERE username = NEW.player;
END;

```



```

CREATE TRIGGER deleteOnlyUnreferencedWords
BEFORE DELETE ON source
FOR EACH ROW
WHEN EXISTS (
    SELECT *
    FROM challenge
    WHERE source = OLD.id
)
BEGIN
    UPDATE source
    SET path = NULL
    WHERE id = OLD.id;

    DELETE FROM word
    WHERE source = OLD.id
        AND (token, source) NOT IN (
            SELECT word, source
            FROM challenge
        );

    SELECT RAISE(IGNORE);
END;

```

3 Strutturazione del codice

3.1 Divisione in package globale

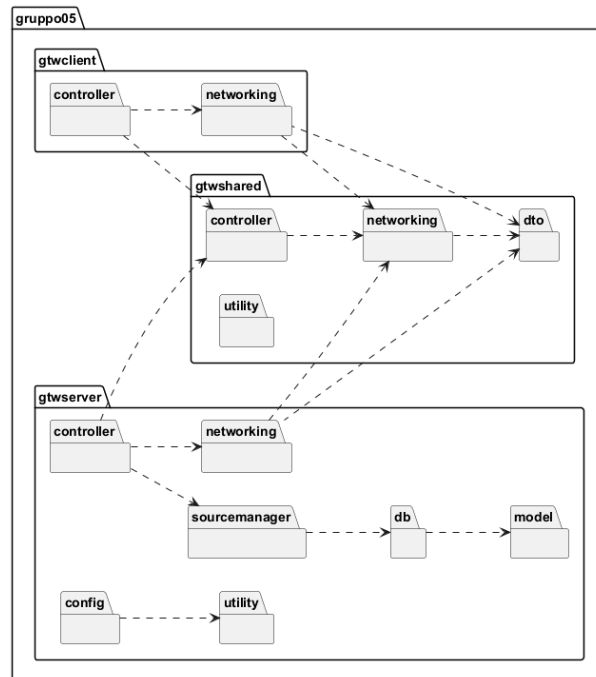


Diagramma dei package generale

Per l'ingegnerizzazione e lo sviluppo del sistema applicativo è stata adottata una configurazione basata su un progetto **Maven multi-modulo**. Questa scelta architetturale favorisce una netta separazione delle responsabilità (*Separation of Concerns*), semplifica la gestione delle dipendenze e ottimizza il ciclo di build, suddividendo il codice sorgente in tre macro-moduli indipendenti ma interconnessi attraverso il modulo **gtwshared** che astrae le caratteristiche condivise, basandosi sul principio di buona progettazione DRY (Don't Repeat Yourself).

Come illustrato nel *Diagramma dei package generale*, l'intero ecosistema risiede sotto lo stesso pacchetto di primo livello denominato **gruppo05**. All'interno di quest'ultimo, l'alberazione si dirama in tre package di secondo livello che identificano univocamente i rispettivi moduli Maven:

- **gtwshared**: Rappresenta un modulo trasversale condiviso, concepito per contenere le strutture dati comuni, le utility globali e i *Data Transfer Object* (**dto**). Questo pacchetto definisce i contratti e i messaggi scambiati lungo il canale di comunicazione, garantendo il forte disaccoppiamento tra client e server ed evitando la duplicazione del codice.
- **gtwclient**: Contiene lo strato software deputato alla gestione dell'interfaccia utente e delle interazioni logiche lato client. Al suo interno, il package **controller** ospita i componenti di controllo delle *view*, i quali svolgono il ruolo di intermediari tra l'interazione diretta dell'utente finale e lo strato di rete (**networking**), traducendo gli input in comandi per la logica applicativa.
- **gtwserver**: Costituisce il nucleo di backend del sistema, progettato per gestire le sessioni di gioco concorrenti dei due client connessi e per coordinare le dinamiche globali della partita. Oltre ai moduli di controllo e di rete, include il package **sourcemanager**, un componente specializzato nell'estrazione e nell'analisi di sorgenti testuali per la formulazione dinamica dei quesiti di gioco (parole criptate). La persistenza e la modellazione del dominio sono delegate ai relativi sotto-package **db** e **model**.

L'architettura così strutturata permette una comunicazione pulita ed efficiente, abilitando la logica di gioco competitiva in cui i client si collegano al server centrale per risolvere gli indovinelli testuali generati in tempo reale.

3.2 Organizzazione di GTWShared

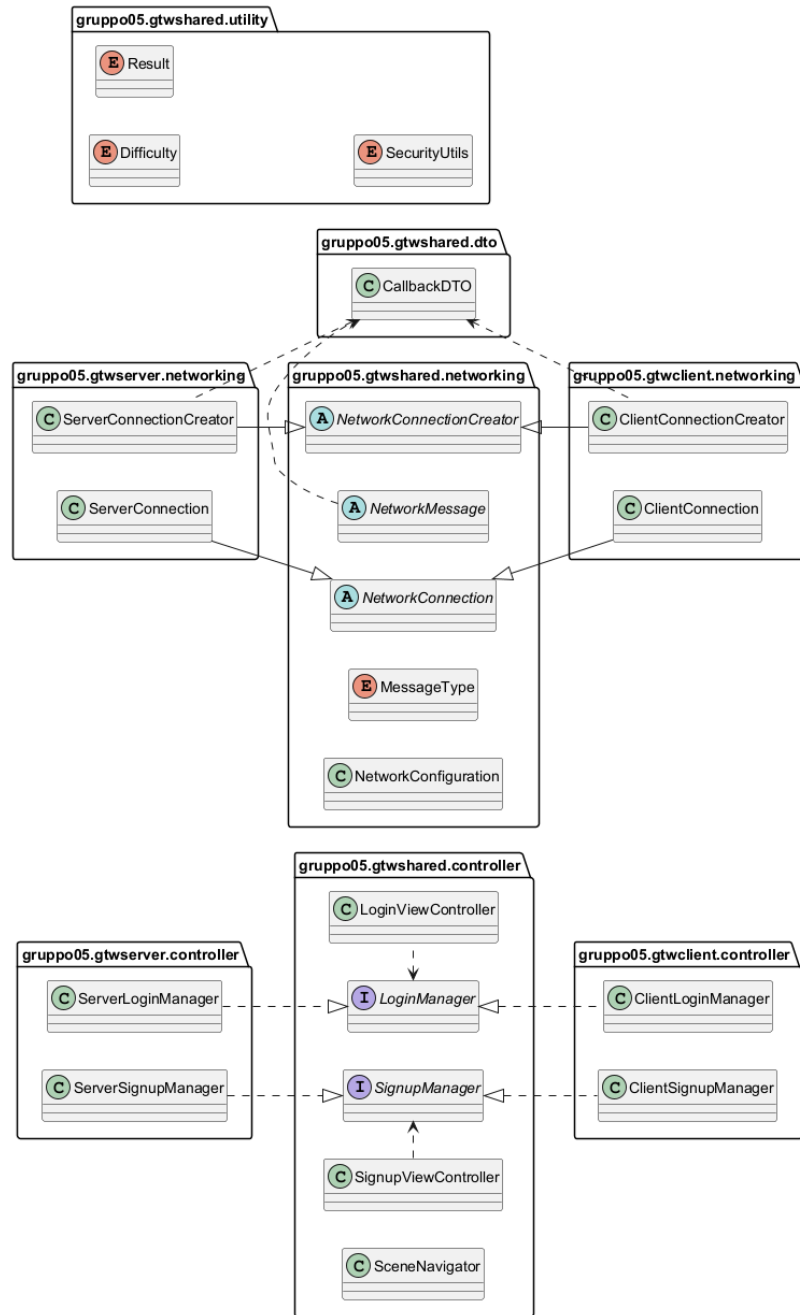


Diagramma delle classi per il modulo condiviso (GTWShared)

Architettura del modulo condiviso (gtwshared)

L'architettura è progettata per garantire una comunicazione bidirezionale, asincrona e thread-safe, sfruttando i socket TCP nativi di Java e la serializzazione degli oggetti tramite `ObjectOutputStream` e `ObjectInputStream`.

Il sistema si disaccoppia nettamente dall'interfaccia grafica (UI) mediante l'uso combinato del pattern *Builder* e di un Data Transfer Object (DTO) centralizzato ed immutabile. Questo previene stati inconsistenti nella vista e standardizza il transito dei messaggi legati al ciclo di vita dell'applicazione (autenticazione, matchmaking, dinamiche della partita, statistiche e storico).

Nel diagramma delle classi complessivo (strutturato nei package `gruppo05.gtwshared.utility`, `dto`, `networking` e `controller`), vengono definite le specifiche componenti descritte di seguito.

1. CallbackDTO

Descrizione generale: La classe `CallbackDTO` (Data Transfer Object) è un costrutto orientato al trasporto sicuro dei dati dal layer di rete (modello) al layer di presentazione (interfaccia utente). Si tratta di un oggetto di trasferimento dati flessibile, progettato specificamente per fungere da collegamento di comunicazione standardizzato tra il livello di rete e l'interfaccia utente (UI Layout). Implementa il pattern creazionale *Builder* per consentire la materializzazione di istanze totalmente immutabili di sola lettura (*read-only*).

Funzionalità e Logica di Business: Nelle applicazioni di rete accoppiate ad interfacce grafiche reattive, il rischio principale è la presenza di oggetti parzialmente configurati o instabili. `CallbackDTO` risolve il problema aggregando tutti i possibili campi di rete (credenziali, dati della lobby, dati della partita in corso, metriche dello storico) ma esponendoli attraverso un'unica interfaccia pulita. Le variabili interne vengono popolate selettivamente solo se strettamente necessarie in base al `MessageType` ricevuto dal server, lasciando i restanti campi a valori neutri o `null` senza compromettere l'integrità dell'oggetto.

Struttura Interna: All'interno di `CallbackDTO` è definita la classe statica innestata `MatchRecord`, la quale funge da record cronologico immutabile per rappresentare una singola partita conclusa all'interno dello storico utente. Tale struttura memorizza l'username dell'avversario sfidato, l'esito finale della partita — espresso mediante l'enumerazione `Result` (con valori `WIN` e `LOSE`) —, il timestamp esatto dell'evento tramite `LocalDateTime`, nonché il tempo di risposta personale espresso in millisecondi.

Inoltre è presente una seconda classe interna, ovvero la classe statica `Builder`, che rappresenta il cuore del pattern creazionale. Contiene metodi setter a cascata (es. `success()`, `gameStartData()`, `historyData()`) che restituiscono l'istanza corrente del builder (`this`) e si concludono con la chiamata `build()` per generare l'oggetto definitivo.

Metodi Principali e Costruttori:

- `private CallbackDTO(Builder builder)`: Il costruttore è privato, imponendo la creazione dell'oggetto esclusivamente tramite il metodo `build()` del `Builder` interno. Copia in blocco tutti i parametri configurati.
- Metodi Getter pubblici: Forniscono l'accesso in sola lettura a tutti i campi (es. `getEventType()`, `isSuccess()`, `getCipheredText()`, `getMatchHistory()`).

2. MessageType

Descrizione generale: Un costrutto di tipo enumerativo (`enum`) che definisce formalmente il dizionario dei messaggi ammessi dal protocollo applicativo di *GuessTheWord*.

Funzionalità e Flusso di Rete: Il costrutto `MessageType` agisce come il protocollo di comunicazione formale dell'intera infrastruttura. Assegna un'etichetta univoca e immutabile ad ogni pacchetto in transito, garantendo che l'integrità del messaggio sia preservata durante il viaggio sul socket TCP.

Questa tipizzazione rigorosa consente al sistema di routing, implementato sia sul lato Client che sul lato Server, di smistare il payload in ricezione in modo efficiente. Attraverso l'uso di costrutti `switch-case` ottimizzati, il sistema indirizza istantaneamente il messaggio al controller specifico competente per la logica di business richiesta, come la gestione del login, le fasi attive della partita o le procedure di disconnessione controllata.

`MessageType` rappresenta quindi il discriminante fondamentale e il punto di sincronizzazione centrale del software. In fase di ricezione, il motore di rete interroga immediatamente questo valore per decidere se inoltrare i dati verso i servizi di backend per l'elaborazione dei dati o verso i controller della UI per l'aggiornamento in tempo reale dell'interfaccia grafica, assicurando una reattività ottimale del sistema.

I tipi di messaggio sono suddivisi logicamente in 6 macro-aree, dettagliate nella Tabella 6.

Tabella 6: Suddivisione logica del protocollo `MessageType`

Categoria Logica	Valori dell'Enum	Descrizione
1. Autenticazione e Registrazione	<code>LOGIN_REQUEST</code> , <code>LOGIN_RESPONSE</code> , <code>REGISTER_REQUEST</code> , <code>REGISTER_RESPONSE</code>	Coordina i processi di validazione delle credenziali, la creazione di nuovi profili utente, interfacciandosi direttamente con il database per garantire la sicurezza degli accessi e delle sessioni.
2. Matchmaking	<code>PLAY_REQUEST</code> , <code>PLAY_RESPONSE</code>	Regola le logiche di abbinamento tra i giocatori. Gestisce l'inserimento degli utenti in code di attesa e notifica in tempo reale le transizioni di stato (<code>waiting/match found</code>).
3. Ciclo di Partita	<code>GAME_START</code> , <code>ANSWER_SUBMISSION</code> , <code>GAME_RESULT</code> , <code>OPPONENT_ANSWERED</code> , <code>WRONG_ANSWER</code>	Amministra l'intera evoluzione della sfida: dal trasferimento iniziale del payload di gioco, alla ricezione dei tentativi di risoluzione, fino alla decretazione del vincitore. Prevede l'invio di segnali asincroni per notificare le azioni avversarie e la segnalazione di tentativi errati.
4. Gestione Connessione	<code>OPPONENT_DISCONNECTED</code> , <code>CLIENT_DISCONNECT</code>	Intercetta e gestisce in sicurezza sia le disconnessioni volontarie (<code>graceful shutdown</code>) sia le cadute di rete anomale causate da instabilità del livello TCP.
5. Storico e Statistiche	<code>HISTORY_REQUEST</code> , <code>HISTORY_RESPONSE</code>	Gestisce la richiesta e l'estrazione dei dati dal DB per fornire lo storico del giocatore.
6. Utility / Debug	<code>TEXT_MESSAGE</code>	Transito di messaggi testuali generici, notifiche o comandi da terminale/popup di debug.

3. NetworkConfiguration

Descrizione Generale: Un contenitore di dati immutabile (*data class*) finalizzato alla centralizzazione dei parametri di rete essenziali all'instaurazione di un canale di comunicazione socket.

Funzionalità: Centralizza e rende sicuri i parametri necessari per l'apertura dei socket. Viene tipicamente istanziata durante la fase di bootstrap dell'applicazione dopo la lettura dei file di configurazione esterni. Dispone inoltre di un costruttore semplificato dedicato alla configurazione del solo ascolto locale lato Server, che richiede esclusivamente la porta di rete, omettendo l'indirizzo IP.

4. NetworkConnection

Descrizione Generale: Questa classe astratta si colloca ad un alto livello di astrazione per modellare e gestire una connessione di rete multi-canale. Costituisce il cuore operativo della connettività nel sistema, garantendo un'architettura interamente asincrona e sicura per l'esecuzione concorrente (*thread-safe*).

Funzionalità: Nasconde la complessità della concorrenza (*multithreading*) e dell'I/O di rete. Gestisce la creazione dei socket, il mantenimento dei canali di comunicazione, l'invio sicuro dei dati (*unicast* e *broadcast*) e l'intercettazione degli eventi di rete (ricezione messaggi, cadute di connessione). La classe implementa un robusto meccanismo di tolleranza ai guasti all'interno del metodo `connect()`: i thread di setup eseguono un ciclo di connessione persistente che, in caso di fallimento o disconnessione, attende 5 secondi prima di ritentare automaticamente. L'introduzione del flag `hasFailed` permette di intercettare le riconessioni riuscite e attivare in modo puntuale la callback dedicata `onReconnected(channelIndex)`.

Dettagli Tecnici e Gestione Concorrenza: All'interno della classe è presente una collezione di thread gestori, i cui elementi sono istanze di `ConnectionThread`, una classe interna innestata dedicata alla gestione delle singole connessioni. Poiché i canali di comunicazione operano in parallelo su thread dedicati, la lista è resa thread-safe mediante il wrapper `Collections.synchronizedList`.

La classe applica rigorosamente il pattern architetturale *Template Method*: definisce lo scheletro dell'algoritmo di setup della connettività nel metodo `connect()`, ma delega l'effettiva creazione fisica del socket (Client-side o Server-side) e il conteggio dei canali attesi alle sottoclassi concrete tramite i metodi astratti:

- `protected abstract Socket createSocket() throws IOException;`
- `protected abstract int expectedChannels();`

Fornisce inoltre utili metodi di diagnostica real-time (es. `getActiveChannelCount()`, `areAllChannelsReady()` e `isChannelActive()`) per monitorare lo stato effettivo dei pool di thread.

La logica dei Thread e la prevenzione del Deadlock: All'interno della classe interna `ConnectionThread` (estensione nativa di `Thread`), viene eseguito il ciclo vitale di ascolto continuo. Un dettaglio fondamentale risiede nell'inizializzazione degli stream I/O all'interno del metodo `run()`: `ObjectOutputStream` e `ObjectInputStream`.

L'architettura impone l'apertura dell'`ObjectOutputStream` (oos) prima dell'`ObjectInputStream` (ois). La creazione di un `ObjectInputStream` è un'operazione bloccante che attende la ricezione dell'header di serializzazione dal nodo di rete remoto: se entrambi i nodi di rete (Client e Server) tentassero di aprire per primo l'OIS, l'applicazione cadrebbe in un *deadlock* infinito. L'ordine garantito previene questo stallo.

API Pubblica: Inoltre la classe espone un'API pubblica formata da metodi di trasmissione (`sendTo()`, `broadcast()` e `broadcastExcept()`) che gestiscono l'invio dei dati iterando sui thread attivi, con gestione incapsulata e trasparente delle eccezioni di I/O.

5. NetworkConnectionCreator

Descrizione generale: Classe astratta delegata al controllo della persistenza della configurazione e alla fabbricazione del motore di rete.

Funzionalità di Business: Garantisce la robustezza all'avvio leggendo i parametri di rete (IP e porta) da un file `.properties`. Se il file è assente, ne autogenera uno con valori di default (127.0.0.1 : 5050) per assicurare l'avvio dell'applicativo. In caso di errori critici o file corrotto, mostra un **Alert** grafico e termina l'applicazione in sicurezza. Infine, delega la creazione fisica della connessione alle direttamente sottoclassi tramite il metodo astratto `factory createConnection()`.

6. NetworkMessage

Descrizione generale: Classe base astratta e immutabile che stabilisce il contratto fondamentale per qualsiasi tipo di payload trasportato sulla rete dell'infrastruttura. Implementa l'interfaccia `Serializable`.

Ruolo nel Flusso e Polimorfismo: Ogni pacchetto scambiato tra Client e Server deve estendere questa classe. Stabilisce il contratto base del protocollo: ogni messaggio viaggiante deve sapersi auto-identificare, deve portare con sé il proprio timestamp di genesi temporale e deve essere capace di auto-tradursi in un `CallbackDTO` per la presentazione grafica.

Obbliga tutte le Sottoclassi ad implementare il metodo astratto `public abstract CallbackDTO toDTO();`. Questo schema permette ai ricevitori di rete di trattare i messaggi in modo polimorfo e generico, convertendoli agevolmente nel DTO interpretabile dall'interfaccia grafica.

Analisi delle Sottoclassi Interne Statiche: Ciascuna delle sottoclassi interne statiche di `NetworkMessage` rappresenta una specifica tipologia di messaggio scambiabile sul canale di rete:

- Le classi `LoginRequest` e `LoginResponse` gestiscono la sicurezza degli accessi e l'inizializzazione della sessione utente, trasferendo le credenziali per la verifica e restituendo l'esito dell'autenticazione unitamente al flag relativo ai privilegi amministrativi (`isAdmin`).
- Le classi `RegisterRequest` e `RegisterResponse` costituiscono i componenti speculari al login: incapsulano i dati per l'inserimento di un nuovo account nel database, gestendo le procedure di salvataggio e le eventuali eccezioni sui vincoli di integrità (es. username duplicati).
- Le classi `PlayRequest` e `PlayResponse` coordinano la fase di matchmaking, consentendo al client di richiedere la partecipazione a una partita con una specifica difficoltà e al server di aggiornare asincronamente lo stato dell'utente in lobby.
- La classe `GameStart` rappresenta un comando unidirezionale emesso dal server per avviare la partita: forza lo switch visivo sui client e sincronizza i dati di partenza, ovvero il testo cifrato, il timer e l'identità dell'avversario.
- La classe `AnswerSubmission` è il pacchetto inviato dal client durante lo svolgimento della sfida per trasmettere il tentativo di risoluzione — o il segnale di timeout — abbinato all'esatto tempo di reazione misurato in millisecondi.
- La classe `GameResult` è il messaggio conclusivo spedito dal server per distribuire i verdeti definitivi della partita (vittoria, sconfitta o pareggio), svelare la parola corretta originaria e comunicare l'identità del vincitore.
- Le classi `OpponentAnswered`, `OpponentDisconnected` e `ClientDisconnect` sono segnali di controllo impiegati per monitorare i canali attivi: gestiscono le chiusure controllate dei socket, le anomalie di rete e la notifica in tempo reale delle azioni avversarie.
- Le classi `HistoryRequest` e `HistoryResponse` sono classi di scambio dati finalizzate all'estrazione e alla visualizzazione delle metriche prestazionali storiche dell'utente.
- `TextMessage` è un pacchetto di utility disaccoppiato che consente il transito di stringhe di testo grezze, impiegato principalmente per log di rete, troubleshooting o per l'iniezione dinamica di alert e pop-up nell'interfaccia utente.

3.3 Organizzazione di GTWServer

3.3.1 Organizzazione generale

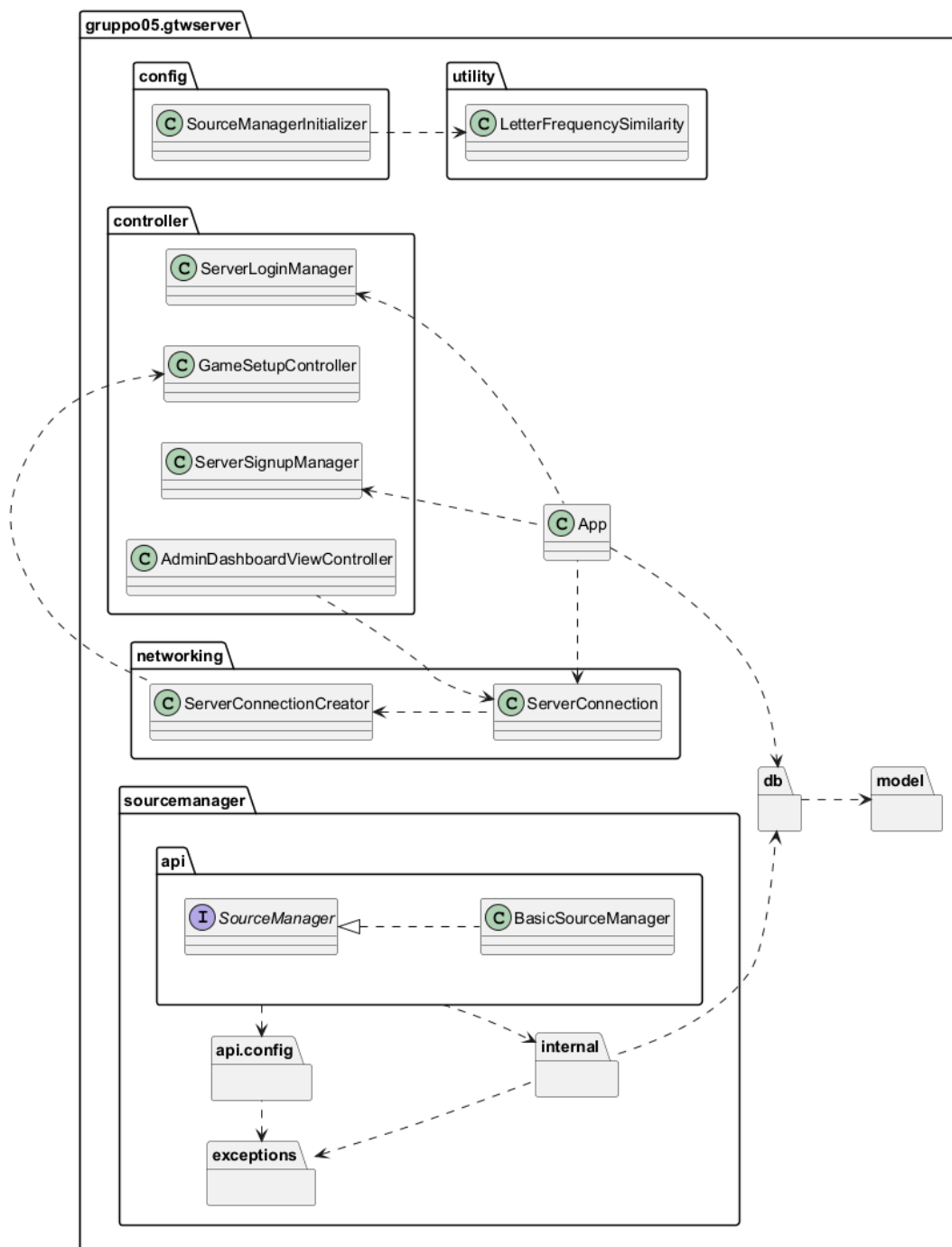


Diagramma delle classi per il modulo del server (GTWServer)

Strato di Presentazione Server (Controller UI)

Lo strato di presentazione del server è strutturato seguendo il pattern architetturale *Model-View-Controller* (MVC). I componenti grafici, sviluppati mediante il framework JavaFX, gestiscono i flussi operativi e le interazioni utente sul nodo, disaccoppiando la logica visuale dalle sottostanti componenti di rete e persistenza.

1. AdminDashboardViewController

Descrizione Generale: La classe `AdminDashboardViewController` è il controller JavaFX dedicato alla gestione del pannello di controllo dell'amministratore (*Server Admin Dashboard*) sul nodo `Server`. Essa assolve la funzione di interfaccia amministrativa centralizzata, offrendo gli strumenti necessari per la manutenzione delle fonti testuali e per il monitoraggio della classifica globale dei giocatori.

Funzionalità e Logica di Business: Le funzionalità esposte dal componente coprono le seguenti aree operative:

- **Gestione delle Sorgenti Documentali:** Consente il caricamento asincrono di nuovi file di testo (in formato `.txt`) attraverso il selettore di file nativo del sistema operativo, fornendo un feedback visivo sullo stato di avanzamento delle attività di parsing ed estrazione lessicale. Supporta inoltre l'eliminazione logica e fisica delle sorgenti, propagando la cancellazione sul database.
- **Monitoraggio del Traffico di Rete:** Sfrutta meccanismi *event-driven* registrandosi ai callback di `ServerConnection` e `ServerConnectionCreator`. Intercetta in tempo reale gli eventi di connessione e disconnessione dei client, aggiornando dinamicamente il contatore delle sessioni attive a schermo.
- **Visualizzazione della Leaderboard:** Interroga il layer di persistenza tramite il componente `ConcretePlayerDAO` per popolare una tabella interattiva contenente le metriche aggregate degli utenti registrati, incluse le partite complessive giocate e il computo delle vittorie.

Integrazione nel Sistema: Il controller agisce come punto di raccordo diretto verso il modulo `SourceManager`, al quale delega i task asincroni di analisi e parsing testuale. Inoltre, garantisce il disaccoppiamento dagli eventi di rete eseguendo i flussi di aggiornamento dell'interfaccia grafica all'interno del thread principale di JavaFX mediante l'invocazione del metodo `Platform.runLater()`.

Architettura di Rete Server (gtwserver)

L'infrastruttura di backend del sistema è ingegnerizzata per gestire la concorrenza, la persistenza dei dati e la logica di business core del gioco. Basandosi sui componenti di connettività astratti, lo strato server specializza le interazioni per coordinare l'accesso simultaneo di più client e mediare l'integrazione con il database relazionale.

1. ServerConnection

Descrizione: La classe `ServerConnection` costituisce l'implementazione concreta lato server dell'infrastruttura di rete. Estendendo la classe base `NetworkConnection`, specializza il comportamento architetturale al fine di accettare e gestire connessioni multiple in ingresso attraverso un'istanza condivisa di `ServerSocket`.

Funzionalità: Il suo compito principale consiste nel porsi in ascolto sulla porta di comunicazione configurata e governare l'intero ciclo di vita delle connessioni in entrata. Poiché il sistema prevede sessioni di gioco competitive in modalità 1 contro 1, il server è istruito per accettare un numero predefinito e vincolato di connessioni (nello specifico, due client) per volta, prima di poter decretare la rete come "pronta" per l'avvio della partita. Questa implementazione sovrascrive il ciclo di setup e avvia un unico thread (*ServerAcceptLoop*) che gira in loop continuo: accetta i client, riassegna dinamicamente gli slot liberati tramite la logica `nextAvailableIndex()` e gestisce le disconnessioni senza perdere la capacità di accogliere nuove richieste.

2. ServerConnectionCreator

Descrizione e Funzionalità: La classe `ServerConnectionCreator` rappresenta il nucleo direzionale e logico dell'infrastruttura di backend. Essa estende la classe astratta `NetworkConnectionCreator` e ricopre simultaneamente il ruolo di *Factory* per l'istanziamento della rete e di *Dispatcher* universale (smistatore) per tutte le richieste di

business in entrata. Elabora i messaggi dei client, converte in DTO e gestisce le operazioni di autenticazione, la coda di matchmaking, l'avvio partita, la validazione e l'interrogazione dello storico.

Struttura Interna e Gestione Concorrenza (Thread-Safety): Poiché il server processa richieste parallele provenienti da client distinti operanti su thread concorrenti, la classe impiega strutture dati *thread-safe*, tra cui una `ConcurrentHashMap` per tracciare in tempo reale gli utenti autenticati (`loggedUsers`) e le sessioni attive (`activeGames`). Impiega inoltre un mutex esplicito denominato `matchLock`, necessario per prevenire *race condition* critiche durante la fase di accoppiamento dei giocatori in lobby.

Adotta inoltre la Dependency Injection per incapsulare il `SourceManager` globale. Gestisce in modo asincrono i fallimenti durante la creazione della partita invocata dal `GameSetupController`, intercettando errori dalla callback dedicata e inoltrando il comando testuale `ERROR_REDIRECT`: ai client interessati per ripristinare la coda ed annullare il match corrotto in modo sicuro.

3. GameSetupController

Descrizione: La classe `GameSetupController` funge da strato di astrazione e disaccoppiamento tra l'infrastruttura pura di rete e la logica di business procedurale del gioco. Agisce come il motore decisionale centrale incaricato di orchestrare l'inizializzazione e la configurazione dinamica di ogni sessione di sfida. La classe delega la complessa generazione testuale a un gestore centralizzato, focalizzandosi sul coordinamento del flusso operativo.

Funzionalità: Il componente `GameSetupController` gestisce e risolve i conflitti nel caso in cui i due partecipanti scelgano livelli di difficoltà differenziazioni durante la fase di matchmaking, applicando un criterio di selezione casuale con una distribuzione di probabilità del 50%. Una volta definita la difficoltà della partita, la classe determina i parametri operativi della sessione — inclusi la durata del timer, la complessità lessicale, la frequenza massima dei termini ammessi e l'offset crittografico del cifrario — coordinando contestualmente le interrogazioni al database per il recupero dei testi sorgente. Il processo si conclude con la generazione del quiz cifrato destinato alla risoluzione da parte dei client.

Struttura Interna: Il costruttore della classe adotta il pattern architetturale della *Dependency Injection*, ricevendo in ingresso un'istanza centralizzata del componente `SourceManager`. In questa fase, il costruttore provvede inoltre all'inizializzazione dei campi interni mediante valori di *fallback* predefiniti. Questa scelta di design assicura il mantenimento di uno stato coerente dell'applicazione e previene *crash* sistemici o la generazione di stati nulli (`null`) qualora lo strato di persistenza o il modulo generatore non risultassero temporaneamente raggiungibili.

Metodo Principale: Il metodo principale `generateMatchData(Difficulty, Difficulty, Consumer<String> onError)` coordina la preparazione dei dati di una sessione di gioco, governando una pipeline logica strutturata nelle seguenti fasi:

- **Configurazione Dinamica dei Parametri:** Sfrutta un costrutto di controllo `switch` per determinare a runtime la durata del timer di gioco in base al livello di difficoltà estratto. I parametri temporali sono impostati rispettivamente a 90s per la modalità `EASY`, 60s per la modalità `NORMAL` e 30s per la modalità `HARD`.
- **Estrazione dei Dati dal Dominio:** Recupera il materiale testuale grezzo dallo strato di persistenza istanziando un componente `ConcreteSourceDAO` e seleziona in modalità pseudo-casuale una sorgente (`Source`) tra quelle disponibili a sistema.
- **Generazione Asincrona e Sincronizzazione:** Il controller delega integralmente la creazione e l'offuscamento del testo al componente `SourceManager`, richiamando la relativa funzione e passando come argomenti la sorgente estratta e il livello di difficoltà. Poiché tale elaborazione sottostante si articola mediante callback asincrone di successo e fallimento, il controller impiega un meccanismo di sincronizzazione basato su `CountDownLatch`. Questo costrutto blocca temporaneamente il thread corrente, forzando un'attesa sincrona fino al completamento

dell'elaborazione remota, e introduce un timeout di sicurezza fissato a **15 secondi** per scongiurare condizioni di stallo infinito (*deadlock*).

- **Gestione Strutturata degli Errori:** Qualora si verificano eccezioni di runtime, anomalie nell'accesso al database, violazioni del timeout di generazione o in assenza di fonti valide, la classe intercetta l'anomalia e attiva la callback `onError` fornita programmaticamente dal chiamante, garantendo una notifica controllata e robusta dei guasti.

Componenti Algoritmici di Utility

Il presente capitolo analizza i componenti logico-matematici trasversali del sistema.

1. Indice di Similarità Testuale

Descrizione Generale e Scopo: L'indice di similarità è un componente algoritmico implementato nella classe `LetterFrequencySimilarity`, situata all'interno del pacchetto `gruppo05.gtwserver.utility`. Il suo scopo risiede nella valutazione quantitativa del grado di correlazione o somiglianza strutturale tra due stringhe di testo. Nel contesto del sistema, questa componente (che estende formalmente l'interfaccia funzionale `BiPredicate<String, String>`) viene impiegata per scartare termini lessicalmente troppo vicini ad altri elementi già inclusi nel testo proposto all'utente. Tale filtro previene situazioni in cui la risoluzione del quesito risulti banale a causa della presenza di parole strettamente correlate o derivate posizionate in prossimità del testo cifrato.

Basi Matematiche: Cosine Similarity e Vettori di Frequenza: Il nucleo dell'algoritmo si basa sul concetto matematico di *Similarità del Coseno* (*Cosine Similarity*) applicato alle distribuzioni di frequenza dei caratteri alfabetici. L'elaborazione di una coppia di stringhe si articola in tre fasi sequenziali:

1. **Vettorizzazione:** Ciascuna stringa in ingresso viene normalizzata in caratteri minuscoli (*case-insensitive*). Successivamente, viene istanziato un array di interi a dimensione fissa pari a 26 locazioni, corrispondenti alle lettere dell'alfabeto latino standard. L'indice di aggiornamento dell'array viene calcolato tramite un *offset* basato sulla codifica ASCII, mappando il carattere 'a' all'indice 0 e il carattere 'z' all'indice 25. Durante la scansione della parola, ogni occorrenza di una lettera incrementa il rispettivo contatore. Eventuali caratteri speciali, segni di punteggiatura o lettere accentate esterni all'intervallo [0-25] vengono ignorati in modo trasparente, ottimizzando le prestazioni ed evitando computazioni ridondanti.
2. **Calcolo della Similarità:** I vettori di frequenza A e B così ottenuti vengono confrontati analiticamente calcolando il rapporto tra il loro prodotto scalare e il prodotto delle rispettive norme euclidee:

$$\text{sim}(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|} \quad (1)$$

Il calcolo restituisce un valore continuo normalizzato nell'intervallo [0.0, 1.0], dove 1.0 indica vettori con proporzioni identiche (es. parole identiche o anagrammi perfetti) e 0.0 indica vettori ortogonali e disgiunti (parole che non condividono alcuna lettera).

3. **Valutazione tramite Soglia (*Thresholding*):** Il coefficiente di similarità viene confrontato con una costante di soglia empirica definita a tempo di compilazione (`SIMILARITY_THRESHOLD = 0.80`). Se il valore calcolato uguaglia o supera tale soglia, il predicato restituisce `true` (classificando i termini come "simili"), altrimenti restituisce `false`. L'incapsulamento di tale logica dietro una costante configurabile garantisce un'elevata manutenibilità in vista di future tarature del gameplay.

3.3.2 Organizzazione di SourceManager

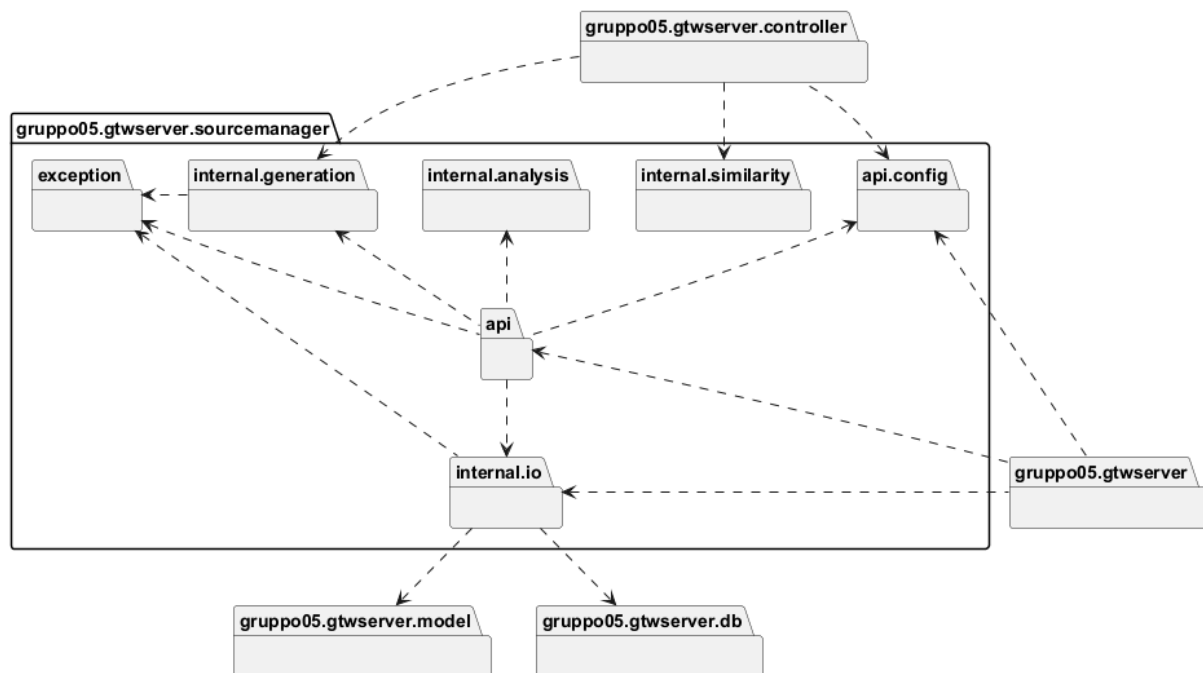


Diagramma dei package per il Source Manager nel modulo del server (GTWServer)

Architettura del Source Manager

Il componente **Source Manager** è il modulo software responsabile della gestione del ciclo di vita e della logica di business associata alle “Fonti” (Sources) all’interno del sistema. Il componente espone un set di funzionalità core progettate per operare in modalità asincrona, ottimizzando le prestazioni ed evitando il blocco del thread chiamante.

Le responsabilità principali del *Source Manager* includono:

- **Persistenza dei metadati:** Gestione e memorizzazione delle informazioni strutturate relative alle fonti all’interno del sistema di storage.
- **Business Logic:** Generazione algoritmica e casuale di domande basate sulle fonti stesse, nello specifico stringhe in formato *testo con parola cifrata* e la corrispondente *parola cifrata da indovinare*.

Al fine di mantenere un’architettura snella e performante, il componente adotta una strategia ibrida per la gestione dei dati: i metadati e le configurazioni principali vengono persistiti, mentre i contenuti voluminosi delle fonti vengono mantenuti ed elaborati direttamente in memoria. Questi ultimi vengono letti dinamicamente dal file system a ogni operazione principale (come l’aggiunta di una risorsa o la generazione di una domanda), evitando così un sovraccarico del database.

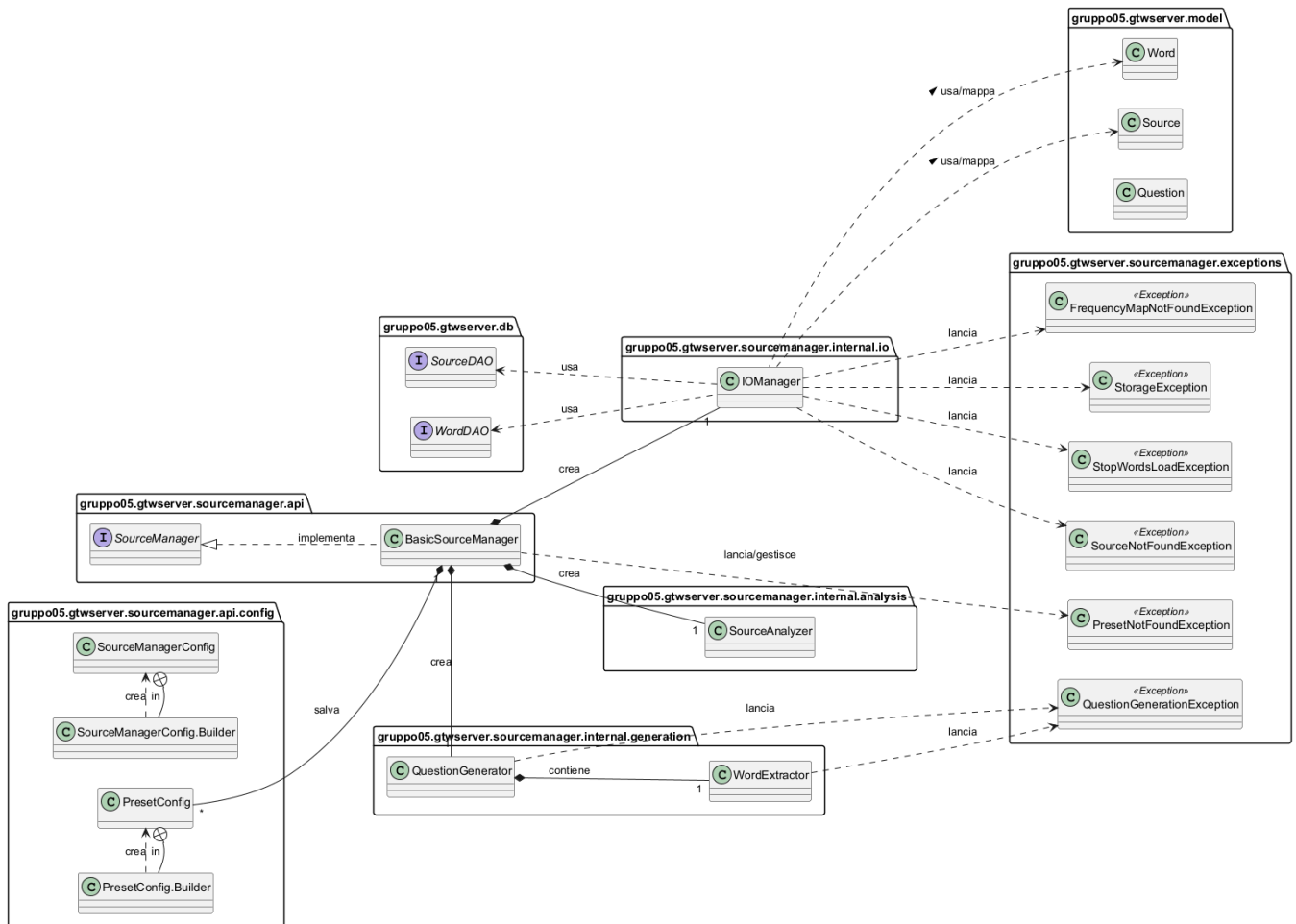


Diagramma delle classi per il Source Manager nel modulo del server (GTWServer)

1. Ecosistema, Interazioni e Stack Tecnologico

L'ossatura tecnologica si basa nativamente sulle librerie standard di Java:

- **Java NIO (New I/O):** Utilizzato per l'accesso efficiente e non bloccante al file system durante la lettura delle fonti in memoria.
- **Concurrency Utilities (ExecutorService):** Impiegato per la gestione del pool di thread dedicati all'esecuzione asincrona delle operazioni di business.
- **Stream API e Functional Interfaces:** Sfruttate per l'elaborazione fluida, dichiarativa e funzionale dei dati in memoria.

Flusso di Interazione

Nel contesto macro del sistema, il *Source Manager* si colloca nel layer di business logic e interagisce come segue:

1. **Chiamanti:** Viene invocato dal layer controller di un'architettura **MVC (Model-View-Controller)** del server, il quale intercetta le richieste utente (es. inserimento/rimozione di una risorsa o richiesta di generazione di una domanda) e le delega al componente.
2. **Dipendenze:** Per l'archiviazione dei dati, il componente è completamente **agnostico rispetto al sistema di storage**. Non si collega direttamente a un database specifico, ma interagisce esclusivamente tramite due interfacce di tipo **DAO (Data Access Object)**, garantendo un elevato disaccoppiamento e la massima manutenibilità.

2. Pattern Architetture e Struttura Interna

Il design interno del *Source Manager* è stato modellato applicando diversi Design Pattern consolidati, al fine di esporre un'interfaccia pulita verso l'esterno e mantenere un'alta configurabilità interna:

- **Facade Pattern:** La complessità interna del componente è mascherata dietro un'unica interfaccia pubblica, *SourceManager* (implementata concretamente da *BasicSourceManager*). Questo garantisce ai team di sviluppo che utilizzano il modulo un punto di accesso centralizzato e semplificato, senza necessità di conoscere i sottosistemi interni.
- **Builder Pattern:** La fase di inizializzazione e configurazione del componente è delegata ad apposite classi *Builder*. Questo approccio risolve il problema dei costruttori telescopici, consentendo l'assegnazione di parametri opzionali e facilitando la creazione di *preset* predefiniti per la generazione delle domande.
- **Callback Pattern:** Per supportare la natura asincrona delle operazioni senza bloccare il Controller MVC chiamante, gli esiti delle operazioni (es. la generazione di una domanda) vengono restituiti al client tramite l'utilizzo di funzioni di callback, garantendo un flusso di esecuzione non bloccante.

3. Principi di Design (SOLID e Best Practices)

L'architettura del sistema riflette una forte aderenza ai principi SOLID e alle best practices dell'ingegneria del software, mirando a un'elevata coesione e un basso accoppiamento:

- **Single Responsibility Principle (SRP):** Il *BasicSourceManager* funge da puro orchestratore. Le responsabilità effettive sono nettamente separate in moduli distinti:
 - *SourceAnalyzer*: Delegato all'analisi delle fonti.
 - *IOManager*: Responsabile dell'interazione con il file system.
 - *QuestionGenerator*: Specializzato nella logica algoritmica di creazione delle domande, il quale a sua volta delega l'estrazione delle parole a un *WordExtractor* dedicato.
- **Dependency Injection (DI) & Dependency Inversion (DIP):** Tutti i moduli centrali ricevono le proprie dipendenze dall'esterno, dipendendo da astrazioni (interfacce) piuttosto che da implementazioni concrete. Questo principio è spinto fino ai livelli più bassi: persino i generatori di casualità (*Randomizers*) vengono iniettati nel *QuestionGenerator* e nel *WordExtractor*. Questa scelta garantisce una **testabilità assoluta**, rendendo possibile il testing deterministico (unit testing via JUnit) tramite mock o seed fissi.
- **Don't Repeat Yourself (DRY):** La creazione del modulo *IOManager* accentra tutta la logica di input/output, eliminando la duplicazione del codice di accesso alle risorse in memoria sparso nel componente.
- **Open/Closed Principle (OCP):** Grazie all'uso pervasivo dell'inversione delle dipendenze e dell'iniezione tramite *Builder*, il sistema risulta intrinsecamente predisposto all'estensione. È possibile, ad esempio, introdurre nuovi algoritmi di estrazione (*WordExtractor*) o nuovi sistemi di storage (tramite i DAO) senza modificare il codice sorgente esistente.

4. Gestione della Concorrenza e Thread Safety

Il *Source Manager* è nativamente predisposto per operare in ambienti multithreading e gestire un elevato throughput di richieste, garantendo affidabilità senza colli di bottiglia causati da eccessiva sincronizzazione. La strategia di concorrenza si basa sui seguenti pilastri:

- **Delega dell'Esecuzione (Task Delegation):** La totalità delle operazioni asincrone è orchestrata delegando i task a un *ExecutorService*. Questo approccio permette di ottimizzare l'uso delle risorse hardware della CPU tramite un pool di thread dedicato, liberando immediatamente il thread chiamante (es. il Controller MVC) e prevenendo il blocco dell'applicazione.

- **Gestione dell'Asincronia e Responsabilità del Chiamante:** Si è scelto deliberatamente di non utilizzare `Task` e `Service` di JavaFX all'interno del componente `SourceManager`, optando invece per un `ExecutorService` standard. Questa decisione risponde a tre pilastri progettuali:
 1. *API Safe-by-Default:* Esponendo un'API nativamente asincrona basata su callback, si azzera il rischio che il programmatore client (il Controller UI) commetta errori di threading capaci di bloccare l'interfaccia grafica.
 2. Incapsulamento delle Risorse Hardware: Il `SourceManager` esegue accessi intensivi al file system tramite Java NIO. Mantenendo un proprio thread pool isolato, il Dominio auto-regola il carico di I/O. Se si delegasse l'asincronia all'interfaccia, i Controller potrebbero generare un numero incontrollato di Task simultanei, causando colli di bottiglia sul disco.
 3. Clean Architecture e Riutilizzabilità: Mantenendo lo strato del Model del tutto svincolato dalle librerie `javafx.concurrent.*`, l'API di business rimane pura e riutilizzabile in qualsiasi altro contesto software (es. un'applicazione console o un backend web), rispettando il principio del disaccoppiamento.
- **Sicurezza Intrinseca tramite Sola Lettura (Read-Only State):** Per quanto concerne l'accesso alle fonti in memoria e su file system (tramite Java NIO), il sistema adotta un paradigma di sola lettura. Poiché il ciclo di vita dei file fisici non prevede modifiche concorrenti a runtime da parte del modulo, l'accesso simultaneo da molteplici thread è intrinsecamente *thread-safe*. L'assenza di *race condition* su queste risorse elimina la necessità di implementare costosi meccanismi di *locking*, massimizzando le prestazioni in fase di estrazione dati e generazione delle domande.
- **Isolamento delle Operazioni di Scrittura:** Le uniche mutazioni di stato all'interno dell'ecosistema (`AddSource` e `DeleteSource`) sono limitate all'inserimento e all'eliminazione dei metadati. Essendo operazioni relegate esclusivamente al database, la responsabilità della sincronizzazione e della gestione delle transazioni concorrenti è volutamente astratta e demandata al layer di persistenza (le implementazioni concrete delle interfacce DAO). Questo garantisce che il *Source Manager* rimanga agnostico e privo di logiche di lock a basso livello.
- **Design Rigorosamente Stateless:** Tutti i componenti di elaborazione interna (come `SourceAnalyzer`, `QuestionGenerator` e `WordExtractor`) sono progettati senza alcuno stato mutabile di istanza. Le informazioni contestuali necessarie durante l'elaborazione dei quiz vengono propagate esclusivamente tramite lo *Stack* dei thread (parametri dei metodi e variabili locali), garantendo l'assenza totale di *race conditions* in ambiente concorrente, pur mantenendo singole istanze condivise dei componenti.

5. Gestione degli Errori ed Eccezioni

Data la natura asincrona del componente, la gestione delle anomalie non si affida al tradizionale meccanismo di propagazione sincrona (`throws`), bensì a un canale di notifica dedicato in caso di fallimento (*onFailure callback*), implementato tramite l'interfaccia funzionale `Consumer<Exception>`.

Il componente mappa i fallimenti interni in un set di eccezioni personalizzate e fortemente tipizzate, permettendo al Controller chiamante di identificare la causa esatta dell'errore e reagire di conseguenza. Le eccezioni sono così suddivise in base al dominio dell'anomalia:

- **Logica di Generazione e Configurazione:**
 - `QuestionGenerationException`: Sollevata in caso di anomalie impreviste durante l'esecuzione dell'algoritmo di generazione delle domande.
 - `PresetNotFoundException`: Lanciata quando le impostazioni di generazione richieste dal client non corrispondono a nessun preset configurato in fase di build.
- **Analisi dei Contenuti e I/O (File System):**
 - `StopWordsLoadException`: Segnala un fallimento durante il caricamento o la lettura del file delle stop-word necessario all'analisi testuale.

- **FrequencyMapNotFoundException**: Identifica l'impossibilità di reperire la mappa delle frequenze delle parole per la fonte specificata.
- **SourceNotFoundException**: Rilevata quando la fonte richiesta (in memoria o su file system) non è presente o non è accessibile.
- **StorageException**: Incapsula gli errori generati dal layer DAO (es. problemi di connessione o vincoli di integrità sul database), isolando l'infrastruttura di persistenza dalla logica di business.

6. Ottimizzazione della Memoria e Gestione delle Risorse

Poiché il componente opera su file testuali potenzialmente estesi, l'architettura adotta strategie mirate per abbattere il footprint di memoria (RAM) e prevenire l'esaurimento delle risorse di sistema (File Descriptors):

- **Euristica di Salto (Zero-RAM Footprint)**: Invece di caricare l'intero contenuto delle fonti in memoria o in liste temporanee, il sistema stima il numero di parole totali analizzando la dimensione in byte del file fisico a livello di sistema operativo. Questo calcolo permette di effettuare un salto randomico (**skip**) posizionale direttamente nello Stream NIO.
- **Consumo Pigro tramite Iterator**: Una volta effettuato il salto, l'elaborazione non consuma l'intero stream residuo, ma sfrutta un **Iterator** per estrarre in modo sequenziale ed esclusivo le sole frasi necessarie alla generazione della domanda (*short-circuiting* manuale), lasciando il resto del file non letto.
- **Sicurezza dei File Descriptor**: Le operazioni di I/O (tramite `Files.lines()`) sono incapsulate all'interno di costrutti.

7. Resilienza e Meccanismi di Fallback (Graceful Degradation)

L'architettura è progettata per tollerare le anomalie e l'imprevedibilità del linguaggio naturale, assicurando la restituzione del servizio (la creazione della domanda) anche in condizioni sub-ottimali:

- **Fallback sull'Estrazione Semantica (WordExtractor)**: Durante la selezione della parola da indovinare, il sistema applica filtri rigidi (stop-words, frequenza massima, similarità). Se un testo è particolarmente atipico e tutti i candidati vengono scartati, il modulo non fallisce; utilizza un sistema di valutazione iterativa basata su un criterio configurabile (`fallbackWordCriterion`) per eleggere e conservare il "candidato meno peggio", garantendo la restituzione di una parola giocabile.
- **Fallback sull'Euristica di I/O (BasicSourceManager)**: Poiché il calcolo del salto casuale sul file si basa su una stima matematica, vi è la remota possibilità che l'Iteratore superi la fine del file utile. L'orchestratore intercetta proattivamente questa specifica anomalia ed esegue un ripristino istantaneo riaprendo lo stream e avviando la lettura forzata dall'indice zero, salvando l'esecuzione della richiesta asincrona.

3.3.3 Organizzazione dell'interfaccia con il database

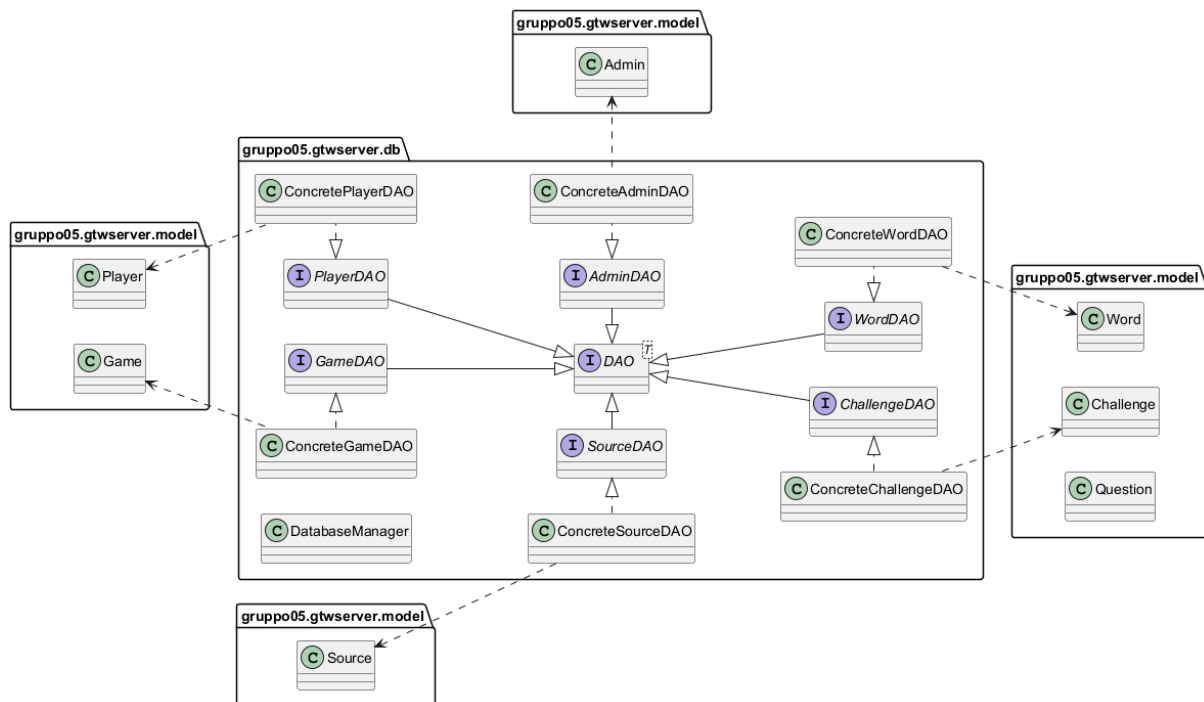


Diagramma delle classi per il model e per il db nel modulo del server (GTWServer)

Strato di Persistenza e Modello del Dominio (db e model)

Lo strato di persistenza del sistema è progettato per garantire la conservazione a lungo termine e l'integrità dei dati di business, isolando completamente le logiche di accesso al database relazionale (*SQLite*) dal resto dell'infrastruttura applicativa. Come evidenziato nel diagramma delle classi, i componenti si suddividono logicamente tra le entità immutabili del dominio e i gestori operativi della persistenza.

1. Il Package model

Il pacchetto `gruppo05.gtwserver.model` ospita gli oggetti del dominio applicativo (tra cui `Player`, `Game`, `Admin`, `Source`, `Challenge` e `Question`). Questi componenti sono implementati come modelli immutabili, garantendo la consistenza dello stato durante le elaborazioni concorrenti del server e prevenendo alterazioni indesiderate al di fuori del controllo transazionale dello strato db.

2. Il Package db e l'Infrastruttura di Persistenza

La gestione dei dati è governata da due attori principali che cooperano per astrarre le primitive SQL: il gestore del ciclo di vita del database e la gerarchia di accesso ai dati.

Il componente `DatabaseManager`: Questa classe funge da punto centralizzato per la configurazione, l'apertura e l'inizializzazione del database relazionale `ServerDB`. Espone due metodi statici fondamentali:

- `getConnection()`: Incapsula la logica di caricamento dei driver JDBC e restituisce un oggetto `Connection` attivo, basandosi su parametri costanti di configurazione interni alla classe stessa.
- `initDB()`: Presiede alla fase di bootstrap del database. Esegue in una singola transazione atomica lo script DDL (*Data Definition Language*) contenente tutte le istruzioni `CREATE TABLE` e `CREATE TRIGGER` necessarie a strutturare ex-novo lo schema relazionale, garantendo l'integrità iniziale del database in caso di prima installazione.

La Gerarchia dei Data Access Object (Pattern DAO): L'accesso alle tabelle applica il *Design Pattern DAO*, specializzandolo per risolvere la variabilità dei modelli di dominio, i quali presentano chiavi primarie differenti (singole o composte) e attributi eterogenei per tipo. La gerarchia è strutturata su due livelli di astrazione:

1. **Interfaccia Generica DAO<T>:** Rappresenta la radice della gerarchia e definisce i contratti per le operazioni macro-scopiche che coinvolgono l'interezza dell'entità o collezioni di essa, astruendo le firme di `select`, `selectAll`, `insert`, `insertAll` e `update`.
2. **Sotto-interfacce Specifiche:** Classi come `PlayerDAO`, `GameDAO`, `AdminDAO`, `WordDAO`, `ChallengeDAO` e `SourceDAO` estendono l'interfaccia radice introducendo i metodi legati alla specificità della chiave del dato (es. `selectById` o `delete`) o query condizionali basate su sottoinsiemi di attributi specifici del dominio.

Integrazione del costrutto Optional: Nelle firme dei metodi di ricerca puntuale all'interno delle sotto-interfacce, il valore di ritorno è incapsulato all'interno di un oggetto `Optional<T>`. L'adozione di questo costrutto a livello architetturale offre due vantaggi cruciali:

- Sradica alla radice il rischio di sollevare eccezioni di tipo `NullPointerException` nelle componenti di business chiamanti.
- Esprime in modo esplicito la possibilità dell'assenza del dato, semplificando la scrittura di codice dichiarativo e sicuro per la verifica della validità dei record prima dell'esecuzione di successive primitive di rete o logiche di gioco.

3.4 Organizzazione di GTWClient

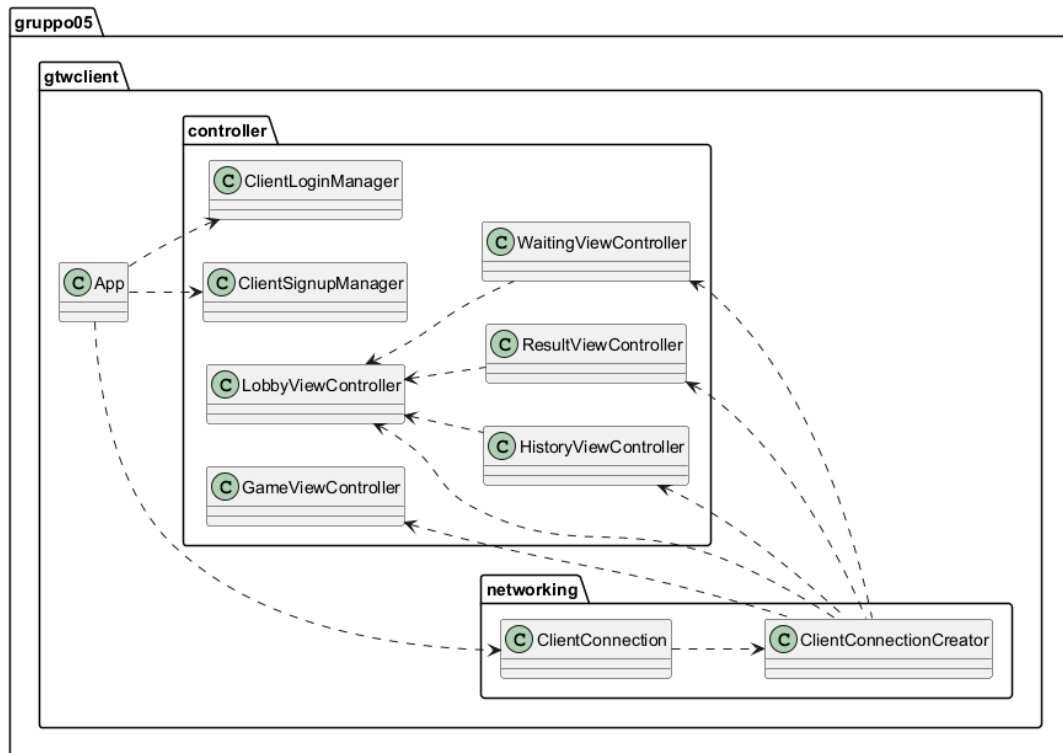


Diagramma delle classi per il modulo del client (GTWClient)

Strato di Presentazione del Client (Controller UI)

Lo strato di presentazione del client è strutturato anch'esso seguendo il pattern architetturale *Model-View-Controller* (MVC). I componenti grafici sono stati sviluppati mediante il framework JavaFX e gestiscono i flussi operativi e le interazioni utente con l'applicazione.

1. GameViewController

Descrizione Generale: La classe **GameViewController** è il controller JavaFX responsabile della gestione del ciclo di vita della partita sul nodo client. Opera in qualità di mediatore diretto tra lo strato di rete locale (**ClientConnection**) e i componenti visuali interattivi offerti al giocatore.

Funzionalità e Logica di Business: Il componente si occupa di coordinare l'evoluzione attiva della sfida attraverso le seguenti logiche:

- **Inizializzazione della Sfida:** All'avvio della sessione di gioco, decodifica i dati di configurazione trasmessi dal server all'interno del **CallbackDTO**, estraendo il testo cifrato da indovinare, l'identificativo dell'avversario e la durata temporale stabilita per il round.
- **Gestione del Timer Asincrono:** Configura una **Timeline** di JavaFX per gestire il countdown visivo in modo asincrono. Presiede alla rilevazione del timeout applicativo: qualora il timer si esaurisca prima dell'inserimento della risposta, invia automaticamente una notifica di sottomissione vuota (scadenza tempo) verso il server backend.
- **Valutazione e Feedback:** Raccoglie i tentativi formulati dal giocatore, misurando localmente i tempi di reazione espressi in millisecondi (**AnswerSubmission**). In caso di errore, provvede a segnalare il fallimento

all'utente e a resettare il campo di testo per i tentativi successivi. Intercetta inoltre i segnali di rete asincroni per informare istantaneamente il giocatore qualora lo sfidante abbia indovinato la parola target.

2. HistoryViewController

Descrizione Generale: La classe `HistoryViewController` è il controller JavaFX preposto alla strutturazione e alla presentazione dello storico dei match disputati dall'utente autenticato. Riceve e mappa i dati estratti dal database centralizzato del server.

Funzionalità e Logica di Business: Il componente organizza le informazioni secondo due schemi di visualizzazione:

- **Rappresentazione delle Statistiche Globali:** Mostra in primo piano metriche riassuntive quali il volume totale di partite giocate, la percentuale di successo (vittorie complessive) e la media aritmetica esatta dei tempi di risposta dell'utente, espressa in millisecondi.
- **Render Tabellare Cronologico:** Popola una tabella interattiva basata sulle istanze di `CallbackDTO.MatchRecord`. Ogni riga esplicita i dettagli di un match concluso, riportando lo username dell'avversario, l'esito finale (WIN/LOSE), il timestamp dell'evento e la rapidità di risoluzione.

La classe supporta infine la navigazione a ritroso, consentendo il rientro guidato alla schermata di lobby tramite l'invocazione del componente `SceneNavigator`.

3. LobbyViewController

Descrizione Generale: La classe `LobbyViewController` costituisce il controller JavaFX delegato alla gestione della schermata principale della Lobby applicativa. In seguito al superamento della fase di login, assume il ruolo di snodo direzionale primario verso tutte le macro-funzionalità del client.

Funzionalità e Logica di Business: Il controller mappa le interazioni utente sui pulsanti della vista traducendole in messaggi formali per il protocollo di rete:

- **Inoltro dei Comandi di Rete:** Gestisce le richieste di estrazione dello storico dei match (`HistoryRequest`) e l'avvio della fase di matchmaking per l'inserimento nel loop competitivo di gioco (`PlayRequest`).
- **Logout e Chiusura Controllata (*Graceful Shutdown*):** Presiede alla pipeline di disconnessione sicura dall'infrastruttura. Al comando di uscita o chiusura, inoltra il messaggio `ClientDisconnect` al server backend, attende il rilascio controllato dei socket locali e termina l'esecuzione dell'applicazione JavaFX in modo pulito.

4. ResultViewController

Descrizione Generale: La classe `ResultViewController` è il controller JavaFX incaricato di governare la fase post-partita sul client, strutturando l'esposizione dei verdetti a seguito del consolidamento dei dati operato dal server.

Funzionalità e Logica di Business: Le responsabilità del modulo si articolano come segue:

- **Rivelazione della Soluzione:** Gestisce la decifrazione visiva finale del quesito, mostrando all'utente la parola target in chiaro indipendentemente dall'esito conseguito durante il match.
- **Risoluzione del Verdetto:** Interpreta i dati aggregati trasmessi dall'evento `GameResult` all'interno del `CallbackDTO` per aggiornare la UI con lo stato finale (vittoria, sconfitta o pareggio), esibendo in concomitanza le metriche temporali della sessione.

Il controller provvede infine alla pulizia delle risorse visive allocate per la sessione prima di consentire il reindirizzamento dell'utente alla lobby principale.

5. WaitingViewController

Descrizione Generale: La classe `WaitingViewController` rappresenta il controller JavaFX preposto alla gestione visiva e logica della fase transizionale di matchmaking, posizionandosi come elemento di intermezzo tra la lobby e l'attivazione della scena attiva di gioco.

Funzionalità e Logica di Business: Il componente implementa le seguenti dinamiche di sincronizzazione:

- **Feedback di Matchmaking in Corso:** Fornisce componenti grafiche animate per notificare all'utente lo stato di attesa attiva di uno sfidante compatibile sul server.
- **Transizioni di Stato Sincronizzate:** Reagisce in modo reattivo alla ricezione asincrona del segnale di protocollo `MATCH_FOUND`. Al rilevamento dell'evento, disabilita immediatamente i controlli di interruzione della ricerca, avvisa l'utente dell'imminente avvio della partita e predispone lo switch automatico di scena.
- **Annullamento della Ricerca:** Fornisce i meccanismi logici per revocare la richiesta di matchmaking prima dell'accoppiamento, notificando il server e riaccompagnando l'utente alla schermata di lobby.

Architettura di Rete Client (gtwclient)

L'infrastruttura lato client è orientata alla gestione di un singolo canale di comunicazione simmetrico verso il backend e alla sincronizzazione asincrona degli eventi di rete con il thread grafico dell'interfaccia utente. Il modulo specializza le classi base per astrarre la complessità dei socket e garantire la fluidità dell'esperienza utente.

1. ClientConnection

Descrizione: La classe `ClientConnection` rappresenta l'implementazione concreta lato client della connessione di rete. Estendendo la classe base astratta `NetworkConnection` (definita nel modulo condiviso `gtwshared`), ne specializza il comportamento architetturale per supportare e vincolare una topologia di tipo 1-a-1 (ovvero un singolo Client accoppiato verso un unico Server).

Funzionalità: Gestisce l'instaurazione, il mantenimento e l'abbattimento di un singolo canale di comunicazione bidirezionale verso il server di gioco. Il suo scopo principale è nascondere al resto dell'applicativo client la complessità della gestione multi-canale ereditata dalla classe padre: operando esclusivamente con un solo server, la classe vincola implicitamente tutte le operazioni di I/O sull'indice di canale fisso 0. Inoltre, sovrascrive il metodo `onReconnected` per avvisare l'utente in tempo reale del ripristino della connessione tramite un Alert grafico generato in sicurezza sul thread JavaFX.

Struttura Interna: Operando esclusivamente con un solo server, la classe vincola implicitamente tutte le operazioni di I/O - sia l'invio dei dati sia la disconnessione - all'indice di canale fisso 0.

Il metodo ereditato `createSocket()` esegue l'handshake TCP attivo istanziando fisicamente il socket di connessione, mentre il metodo `expectedChannels()` restituisce rigorosamente il valore costante 1, impedendo l'apertura di canali aggiuntivi o ridondanti non necessari sul client.

I metodi di supporto `send(Serializable)` e `disconnect()` fungono da *wrapper* semplificati: invocano internamente rispettivamente i metodi `sendTo(0, data)` e `disconnectChannel(0)` della superclasse, offrendo in questo modo ai controller grafici un'API pulita, atomica e priva di indici numerici da gestire esplicitamente.

2. ClientConnectionCreator

Descrizione: La classe `ClientConnectionCreator` funge da *Factory* (fabbrica) e da controllore di rete di massimo livello per l'applicativo client. Essa estende la classe `NetworkConnectionCreator` e si comporta come un componente di mediazione (*Mediator* pattern) strutturato tra il layer di comunicazione TCP grezzo e l'interfaccia grafica utente sviluppata in JavaFX.

Funzionalità: La classe si occupa del bootstrap iniziale dello strato di rete leggendo i parametri di inizializzazione all'interno del file di configurazione locale `client.properties`. La sua funzionalità cruciale risiede nell'agire come *dispatcher* centrale dell'applicazione: intercetta tutti i messaggi asincroni in ingresso dal server, li decodifica in oggetti leggeri di presentazione (`CallbackDTO`) e li smista ai rispettivi controller visivi competenti. Garantisce inoltre la stabilità e la reattività dell'applicazione assicurandosi che qualsiasi aggiornamento o modifica degli elementi della UI avvenga esclusivamente e in modo sicuro sul thread grafico principale di JavaFX.

Struttura Interna: Il metodo `createConnection()`, che costituisce l'override del *Factory Method*, estrae i parametri relativi all'indirizzo IP e alla porta di ascolto, istanzia la classe `ClientConnection` passando il riferimento al metodo locale `handleMessage` come callback di ricezione, e avvia infine la routine di connessione asincrona.